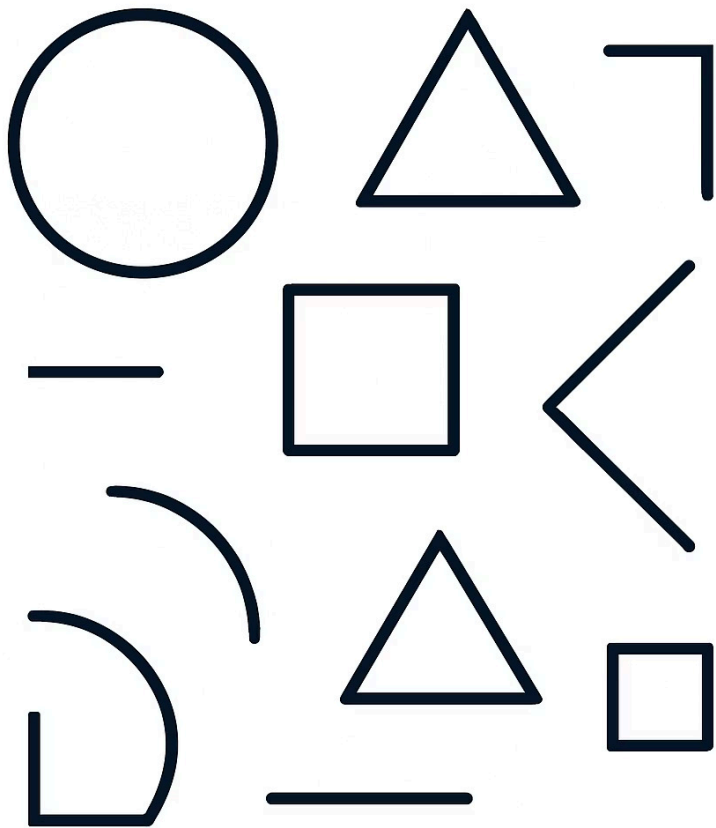




MÓDULO 1 · T2

Fundamentos: JSX y Estilos

🎨 *Componentes, FlexBox y estilos en React Native*

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

Objetivos de la sesión

Al finalizar esta clase serás capaz de construir interfaces nativas completas usando los bloques fundamentales de React Native.

1

JSX con TypeScript

Escribir componentes JSX correctos y tipados en TypeScript, aprovechando el autocompletado y la seguridad de tipos.

2

Componentes base

Dominar `View`, `Text`, `Image` y `TouchableOpacity` como bloques esenciales de cualquier UI.

3

FlexBox nativo

Construir layouts adaptables y responsivos usando el modelo FlexBox propio de React Native.

4

StyleSheet y composición

Crear, combinar y aplicar estilos de forma escalable con `StyleSheet.create` y arrays de estilos.



JSX en React Native

La principal diferencia al llegar desde el desarrollo web es que **no existe HTML** en React Native. No hay `<div>`, `<span>` ni CSS global. Todo se expresa a través de componentes RN específicos.

HTML / Web

```

<p>Pikachu</p>
<div class="card">
  <span>Hola</span>
</div>
```

Etiquetas HTML estándar con clases CSS y selectores globales.

React Native

```
<Image source={{ uri: '...' }} />
<Text>Pikachu</Text>
<View style={styles.card}>
  <Text>Hola</Text>
</View>
```

Componentes nativos con estilos en línea o via `StyleSheet`. Sin CSS externo.

FlexBox en React Native

A diferencia de la web, en React Native **todo View es flex por defecto** y la dirección es `column` (vertical). Cambiar a `row` alinea los hijos horizontalmente.

```
<View style={{
  flex: 1,
  flexDirection: 'row',
  gap: 8
}}>
  <View style={{ flex: 1, backgroundColor: 'red' }} />
  <View style={{ flex: 2, backgroundColor: 'blue' }} />
</View>
```

`flex: 1`

Ocupa todo el espacio disponible en el contenedor padre.

`flexDirection: 'row'`

Dispone los hijos en sentido horizontal en lugar de vertical.

`gap`

Separación automática entre hijos. Disponible desde RN 0.71+.

`justifyContent`

Controla la distribución en el eje principal: `center`, `space-between`, etc.

Propiedades clave de estilo

React Native cubre la mayoría de propiedades visuales de CSS, pero con nombres en *camelCase* y sin unidades (todo en dp). Conocer estas dos familias te permitirá estilizar cualquier componente.

Modelo de caja

`width / height` – dimensiones fijas o porcentuales

`borderRadius` – esquinas redondeadas

`padding / margin` – espacio interior y exterior del elemento

`backgroundColor` – color de fondo del elemento

Texto

`textAlign` – alineación horizontal

`color` – color del texto

`textTransform: 'capitalize'` – transformación

`fontWeight: '700'` – negrita y variantes

`fontSize` – tamaño en dp

Sombras en iOS y Android

Las sombras en React Native se configuran de forma distinta según la plataforma. Para un resultado consistente en ambos sistemas operativos, debes **combinar siempre las dos APIs** en el mismo objeto de estilos.

iOS — shadow* props

```
shadowColor: '#000',  
shadowOffset: {  
  width: 0,  
  height: 2,  
},  
shadowOpacity: 0.12,  
shadowRadius: 4,
```

Controla color, desplazamiento, opacidad y desenfoque de la sombra.

Android — elevation

```
elevation: 3,
```

Un único número que define la altura de la capa. A mayor valor, más sombra y más pronunciada.

 Las propiedades `shadow*` son ignoradas en Android, y `elevation` no funciona en iOS. Usa ambas siempre.

Componer estilos con arrays

Uno de los patrones más poderosos de React Native es pasar un **array de estilos** al prop `style`. Los estilos se fusionan de izquierda a derecha, y el último sobrescribe al anterior en caso de conflicto.

1

Estilo base + variante dinámica

```
<View style={[styles.badge, {  
  backgroundColor: color }]}>
```

Combina un estilo base reutilizable con una propiedad inyectada en tiempo de ejecución.

2

Estilo condicional

```
<View style={[styles.btn, isActive &&  
  styles.btnActive]}>
```

El segundo estilo solo se aplica si la condición es verdadera. Falsy values son ignorados por RN.

3

Composición con variable

```
const cardStyle = [styles.card, isDark  
  && styles.cardDark];
```

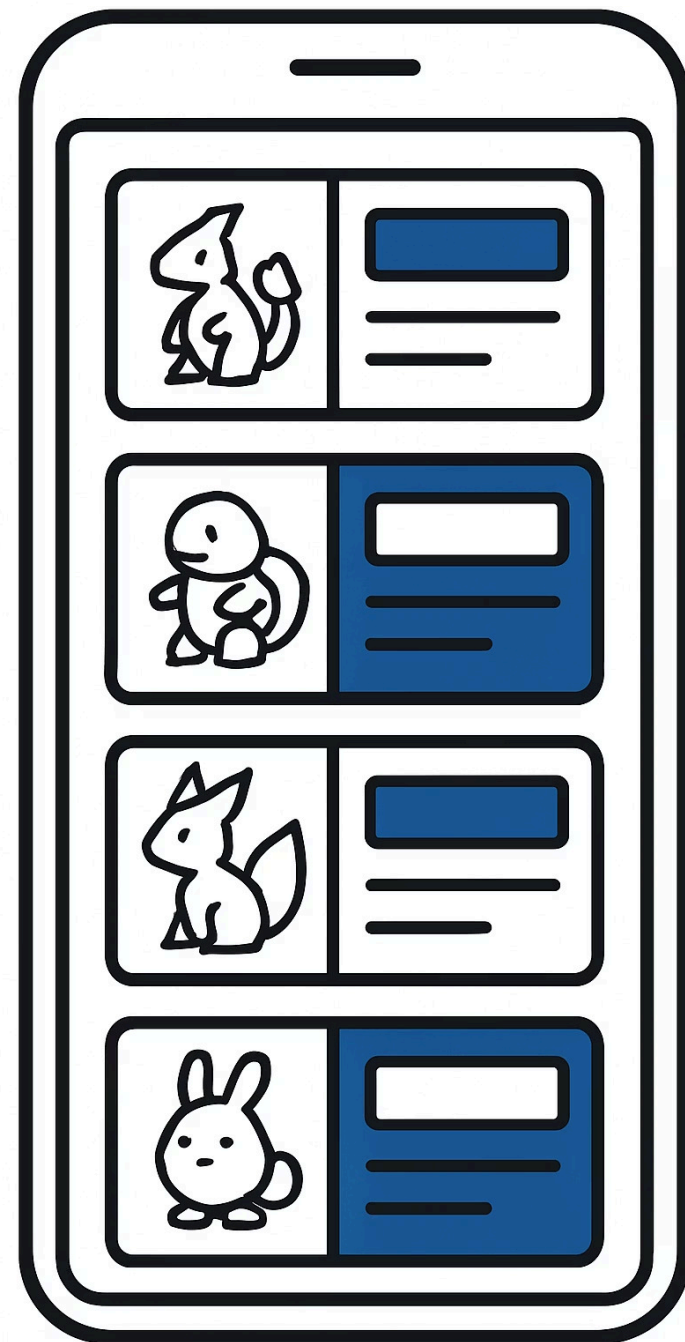
Construye el array en una variable para mantener el JSX limpio y legible.

PokeApp — PokemonListItem.tsx

Este componente pone en práctica FlexBox, imágenes remotas y colores dinámicos en un solo bloque. Cada tarjeta de la lista muestra una franja de color, la imagen del Pokémon y su nombre e índice formateado.

```
<View style={[styles.colorStripe, { backgroundColor: primaryColor }]} />
<Image
  source={{ uri: thumbnail }}
  style={styles.image}
/>
<Text>#{String(indice).padStart(3, '0')}</Text>
<Text style={styles.nombre}>{nombre}</Text>
```

- ❶ La franja de color lateral se logra con un View muy estrecho y flex: 0. El color viene de la API y se pasa como prop. `padStart(3, '0')` formatea el índice como **#001**, **#025**, etc.



Tour Starter — PokemonCard.tsx

Este es el punto de partida del ejercicio. El componente ya tiene la firma de props definida; tu tarea es completar el JSX y los estilos correspondientes a cada sección marcada con `TODO`.

```
// TODO: imagen con source={{ uri: thumbnail }}
// TODO: nombre en Text con fontWeight bold
// TODO: tipo badge con backgroundColor dinámico
export default function PokemonCard({ nombre, thumbnail, tipo }) {
  return (
    <View style={styles.card}>

      </View>
    );
  }

  const styles = StyleSheet.create({
    card: {
      // TODO: añade sombra multiplataforma aquí
      borderRadius: 12,
      padding: 16,
    },
  });
}
```

⚠ No modifiques los props recibidos ni cambies el nombre del componente. Trabaja solo dentro del JSX y del objeto `styles.card`.

Solución — solucion/PokemonCard.tsx

Una implementación completa que cubre imagen, nombre en negrita, badge con color dinámico y sombras multiplataforma. Compara con tu solución antes de continuar.

```
<View style={styles.card}>
  <Image
    source={{ uri: thumbnail }}
    style={styles.image}
  />
  <Text style={{ fontWeight: '700', fontSize: 16 }}>
    {nombre}
  </Text>
  <View style={[styles.tipoBadge, { backgroundColor: badgeColor }]}>
    <Text style={styles.tipoBadgeText}>{tipo}</Text>
  </View>
</View>

// En styles.card:
//  shadowColor: '#000', shadowOffset: { width: 0, height: 2 },
//  shadowOpacity: 0.12, shadowRadius: 4, elevation: 3,
```

- ✓ El color del badge se calcula fuera del componente según el tipo (fuego, agua, planta...) y se pasa como prop `badgeColor` para mantener la lógica separada de la vista.

Ejercicio T02

Abre `starter/PokemonCard.tsx` y completa los cinco puntos para tener un componente funcional y estilizado. Tiempo estimado: **20-25 minutos**.

01

Abre el archivo de inicio

Navega a `starter/PokemonCard.tsx` en tu editor y familiarízate con los props disponibles.

02

Añade el componente Image

Renderiza `<Image source={{ uri: thumbnail }} />` con un tamaño fijo definido en `styles.image`.

03

Muestra el nombre en negrita

Usa `fontWeight: '700'` y un `fontSize` adecuado para que el nombre destaque visualmente.

04

Crea el badge de tipo

Un View con `borderRadius` y `backgroundColor` dinámico recibido como prop. Texto centrado en blanco.

05

Añade sombra al card

Combina `shadow*` para iOS y `elevation` para Android en `styles.card`.

Resumen de la sesión

Estos son los conceptos clave que debes llevarte de T02. Úsalos como referencia rápida mientras construyes tus propios componentes.

Concepto	Elemento / Prop clave	Descripción
View	<code>flex</code> , <code>flexDirection</code>	Contenedor flex fundamental, equivalente al <code><div></code> del web.
Text	<code>fontWeight</code> , <code>fontSize</code>	Todo texto visible debe estar envuelto en un componente <code>Text</code> .
Image	<code>source={{ uri }}</code> o <code>require()</code>	Imágenes remotas vía URL o locales con <code>require</code> .
FlexBox	<code>flexDirection</code> , <code>flex</code> , <code>gap</code>	Sistema de layout por defecto en RN; dirección <code>column</code> por defecto.
Estilos	Arrays <code>[base, variante]</code>	Composición de estilos con fusión de derecha a izquierda.
Sombras	<code>shadowXxx</code> (iOS) + <code>elevation</code> (Android)	Combínalos siempre para soporte multiplataforma.

👉 ¡Bien hecho! En la próxima sesión exploraremos navegación con React Navigation y el paso de parámetros entre pantallas. 🚀